Course Title: Database Management Systems

Course Code: CSE-2423

Course Content of Semester Final Exam

Group-A (20 Marks)

Segment 4. Integrity, Security and Relational Database Design:

Domain constraint, Integrity, Assertions, Triggers, Authorization, Authentication, Security, Privileges, Roles, and Audit trails, Encryption-Decryption Algorithm, Decomposition etc.

Segment 5. Functional Dependency and Normalization:

Functional Dependencies, Closure of a set of Functional dependencies. Un-normal Form (UNF), First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), Boyce and Codd Normal Form (BCNF).

Group-B (30 Marks)

Segment 6. Indexing and Hashing:

Ordered indices, Hash indices, Hash function, Primary index, Secondary index, Dense, sparse, Multilevel indices, B+ tree index files, Handling Bucket Overflows, Overflow Chaining, Closed Hashing, Open Hashing, Linear probing, Hash indices, Dynamic Hashing.

Segment 7. Transaction:

ACID Properties, Transaction state diagram, Implementation of Atomicity and Durability, Shadow copy technique, Concurrent Execution, Serializability, Recoverability, Recoverable schedule, Cascadeless Schedules, Implementation in Isolation, Testing of Serializability.

Segment 8. Concurrency control, Recovery System and Distribute databases:

Lock-Based Protocols, Granting of locks, Two-phase locking protocol, Graph based protocol, Tree protocol, Timestamp based protocols, Deadlock detection and recovery. Failure classification, Storage types, Checkpoints. Distributed data, Replication and Fragmentation.

Segment 4: Integrity, Security and Relational Database Design

---

INTEGRITY CONSTRAINTS

Integrity constraints are **rules enforced by the DBMS** to ensure that the data stored in the database is **accurate, valid, consistent, and meaningful** according to real-world rules.

They prevent invalid data from entering the database and maintain **reliability** of stored information.

Why integrity constraints are important

- Prevent incorrect data insertion
- Maintain consistency between related tables
- Enforce real-world business rules
- Reduce anomalies and errors

Main types of integrity constraints

- Domain constraint
- Referential integrity
- Assertions
- Triggers

---

1. DOMAIN CONSTRAINT

Definition

A **domain constraint** restricts the values that an attribute (column) can take. It ensures that data belongs to a **valid domain**, such as a proper range, format, set, or datatype.

What domain constraints control

- Data type
- Value range
- Allowed set of values
- Format or pattern

Real-life examples

- Age must be between 1 and 120
- Gender must be ‘M’ or ‘F’
- Salary must be positive
- Semester must be one of (‘Spring’, ‘Summer’, ‘Fall’)

SQL Example

```
CREATE TABLE Student (
 sid NUMBER PRIMARY KEY,
 sname VARCHAR2(30),
 age NUMBER CHECK (age BETWEEN 18 AND 40),
 gender VARCHAR2(1) CHECK (gender IN ('M','F'))
);
```

Explanation

- The CHECK constraint enforces valid values.
- Invalid values like age = -5 or gender = 'X' are rejected automatically.

📌 **Exam Tip:**
Domain constraints are **attribute-level constraints** and are enforced during data insertion or update.

---

2. REFERENTIAL INTEGRITY

Definition

Referential integrity ensures that a **foreign key** in one table always refers to an **existing primary key** in another table.

It maintains **consistency among related tables**.

Real-life examples

- A student must belong to an existing department
- A prescription must refer to an existing patient and doctor
- An employee cannot be assigned to a non-existent branch

SQL Example

```
CREATE TABLE Department (
```

```
 did NUMBER PRIMARY KEY,
 dname VARCHAR2(20)
);

CREATE TABLE Employee (
 eid NUMBER PRIMARY KEY,
 ename VARCHAR2(20),
 did NUMBER REFERENCES Department(did)
);
```

Explanation

- did in Employee is a foreign key.
- If department did = 50 does not exist, inserting an employee with did = 50 is rejected.

📌 **Exam Tip:**
Referential integrity prevents **orphan records**.

---

3. ASSERTIONS (CONCEPTUAL CONSTRAINT)

Definition

An **assertion** is a global integrity constraint that specifies a condition that must **always remain true** across the entire database.

Assertions can involve **multiple tables**.

⚠️ **Important:**
Oracle does **not implement assertions**, but they are **academically important and exam-relevant**.

Real-life examples

- An instructor cannot teach more than two courses in the same semester
- Total students in a semester must not exceed 500
- A patient cannot visit two doctors at the same time

Example Schema

Instructor(iid, iname, designation, salary)
Teaches(iid, cid, semester)

Assertion Requirement

“Ensure that an instructor cannot teach more than two courses in the same semester.”

Conceptual SQL Assertion

```
CREATE ASSERTION MaxTwoCourses
CHECK (
 NOT EXISTS (
   SELECT iid, semester
   FROM Teaches
   GROUP BY iid, semester
   HAVING COUNT(cid) > 2
 )
);
```

Explanation

- GROUP BY iid, semester groups courses per instructor per semester
- HAVING COUNT(cid) > 2 detects violation
- NOT EXISTS ensures no such violation occurs

📌 **Exam Tip:**
Always mention that assertions are **conceptual in Oracle** but valid for exams.

---

4. TRIGGERS

Definition

A **trigger** is a stored program that automatically executes when an **INSERT, UPDATE, or DELETE** operation occurs on a table.

Triggers are used to enforce **complex integrity rules** that cannot be handled by simple constraints.

Why triggers are used

- Prevent invalid data entry
- Enforce business rules
- Log unauthorized modifications
- Prevent double booking or conflicts

Example 1: Prevent Negative Salary

```
CREATE OR REPLACE TRIGGER salary_check
BEFORE INSERT OR UPDATE ON Instructor
FOR EACH ROW
BEGIN
 IF :NEW.salary < 0 THEN
   RAISE_APPLICATION_ERROR(-20001,'Salary cannot be negative');
 END IF;
END;
/
```

Explanation

- Trigger fires before insert/update
- Checks the new salary value
- Raises error if salary is negative

Example 2: Hospital – Prevent Double Doctor Visit

Schema

```
Patient(pid, pname, age, gender)
Doctor(did, dname, specialization)
Prescribe(pid, did, drugid, visit_time)
CREATE TRIGGER visit_check
BEFORE INSERT ON Prescribe
FOR EACH ROW
BEGIN
 IF EXISTS (
   SELECT 1 FROM Prescribe
   WHERE pid = :NEW.pid
   AND visit_time = :NEW.visit_time
 ) THEN
   RAISE_APPLICATION_ERROR(-20002,'Patient already assigned');
 END IF;
END;
/
```

📌 **Exam Tip:**
Triggers are **procedural**, **active**, and **event-based**.

AUTHENTICATION vs AUTHORIZATION

Authentication

- Verifies **who the user is**
- Examples: username/password, OTP, biometrics

Authorization

- Determines **what the user is allowed to do**
- Examples: SELECT allowed, DELETE denied

Authentication verifies the identity of a user, whereas authorization determines the actions that the authenticated user is permitted to perform.

PRIVILEGES

Definition

A **privilege** is a permission granted to a user to perform specific operations in the database.

Types of Privileges

1. **Object (Data) Privileges**
2. **System (Database) Privileges**

---

1. Object (Data) Privileges

Common privileges: SELECT, INSERT, UPDATE, DELETE

Example Table

```
CREATE TABLE Student (
 sid NUMBER PRIMARY KEY,
 sname VARCHAR2(30),
 age NUMBER,
 dept VARCHAR2(10)
);
```

SELECT

GRANT SELECT ON Student TO userA;

INSERT

GRANT INSERT ON Student TO userA;
INSERT INTO Student VALUES (4, 'David', 23, 'EEE');

UPDATE

GRANT UPDATE(age) ON Student TO userA;
UPDATE Student SET age = 24 WHERE sid = 2;

DELETE

GRANT DELETE ON Student TO userA;
DELETE FROM Student WHERE sid = 4;

Using VIEW for Controlled Access

CREATE VIEW student_view AS
SELECT sid, sname, age
FROM Student
WHERE dept = 'CSE';

GRANT SELECT ON student_view TO teacher;

---

2. System (Database) Privileges

Used to manage database structure.

Common privileges:

- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CREATE VIEW
- CREATE INDEX

Examples

GRANT CREATE TABLE TO developer;
GRANT ALTER ON Student TO db_assistant;

```
GRANT CREATE VIEW TO analyst;
GRANT CREATE INDEX ON Student TO developer;
```

📌 **Key Difference**

- Object privileges → data access
- System privileges → schema control

ROLES

Definition

A **role** is a named collection of privileges that can be granted to users.

Why roles are needed

- Simplify privilege management
- Assign permissions by job role
- Improve security
- Reduce administrative errors

Hospital Example

```
CREATE ROLE doctor_role;

GRANT SELECT ON Patient TO doctor_role;
GRANT INSERT, SELECT ON Prescribe TO doctor_role;

GRANT doctor_role TO dr_rahim;
```

Revoking Role

```
REVOKE doctor_role FROM dr_rahim;
```

📌 **Note:**
All privileges assigned through the role are removed together.

AUDIT TRAILS

Definition

Audit trails are logs that record:

- Who performed an operation
- What operation was performed
- When it was performed

Used for

- Security monitoring
- Fraud detection
- Compliance
- Accountability

Oracle Example

AUDIT INSERT, UPDATE, DELETE ON Patient;

ENCRYPTION AND DECRYPTION

Encryption

Converting readable data (plaintext) into unreadable form (ciphertext).

Decryption

Converting encrypted data back into readable form.

Purpose

- Protect passwords
- Secure medical/banking data
- Prevent unauthorized access

Conceptual Example

Plain text: HELLO
Encrypted: XJ2259AF

📌 **Note:**
Oracle provides DBMS_CRYPTO for encryption/decryption.

DECOMPOSITION

Definition

Decomposition splits a large relation into smaller relations to reduce redundancy and anomalies.

Goals

- Lossless join
- Dependency preservation
- Remove update anomalies
- Improve logical design

Example

Original:

Employee(eid, ename, dept_name, dept_location)

Decomposed:

Employee(eid, ename, dept_name)
Department(dept_name, dept_location)

☑ EXAM FINAL NOTES

- Assertions → conceptual, multi-table
- Triggers → executable, event-based
- Roles → manage privileges efficiently
- Encryption → data security
- Decomposition → foundation for normalization

Segment 5: Functional Dependency and Normalization

INTRODUCTION TO FUNCTIONAL DEPENDENCY (FD)

What is a Functional Dependency?

A **functional dependency (FD)** is a relationship between two sets of attributes in a relation, where the value of one set **uniquely determines** the value of another set.

Formal Definition

In a relation **R**, an FD
X → Y
means:

If two tuples have the same values for attribute set **X**, then they must have the same values for attribute set **Y**.

Key Idea

- X is called the **determinant**
- Y is called the **dependent attribute**

Real-life Example

Relation:

Student(sid, sname, dept)

FD:

sid → sname, dept

Meaning:

- If sid is known, sname and dept are automatically determined.
- Two students cannot have the same sid but different names.

Important Observations

- Functional dependency is a **semantic constraint**, not a physical one.
- It depends on **real-world meaning**, not just data.
- FDs are used to **design and normalize** databases.

TYPES OF FUNCTIONAL DEPENDENCIES

---

1. Trivial Functional Dependency

An FD **X** → **Y** is trivial if **Y** ⊆ **X**.

Example:

(A, B) → A

Explanation:

- A is already contained in (A, B), so this FD always holds.

📌 Trivial FDs are **always true** and not useful for normalization.

---

2. Non-Trivial Functional Dependency

An FD **X** → **Y** is non-trivial if **Y** ⊄ **X**.

Example:

sid → sname

This is meaningful and important for normalization.

---

3. Completely Non-Trivial FD

If:

X ∩ Y = ∅

Example:

sid → dept

CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES (F^+)

What is Closure?

The **closure of a set of functional dependencies (F^+)** is the **complete set of all functional dependencies** that can be inferred from a given set of FDs using logical rules.

Why Closure is Important

- To find **candidate keys**
- To check **BCNF**
- To test **dependency preservation**
- To validate normalization

ATTRIBUTE CLOSURE (X^+)

Definition

The **attribute closure of X (X^+)** is the set of all attributes that can be determined from X using a given set of FDs.

How to Compute Attribute Closure (Step-by-Step)

Given:

R(A, B, C, D)
FDs:
A → B
B → C

Find A^+

Steps:

1. Start with $A^+ = \{A\}$
2. $A \rightarrow B \Rightarrow$ add $B \rightarrow \{A, B\}$
3. $B \rightarrow C \Rightarrow$ add $C \rightarrow \{A, B, C\}$
4. No more additions

Final:

$A^+ = \{A, B, C\}$

📌 If X^+ contains **all attributes of R**, then X is a **superkey**.

ARMSTRONG'S AXIOMS

These are **inference rules** used to derive new FDs from existing ones.

---

1. Reflexivity Rule

If $Y \subseteq X$, then:

$X \rightarrow Y$

Example:

$(A, B) \rightarrow A$

---

2. Augmentation Rule

If:

$X \rightarrow Y$

then:

$XZ \rightarrow YZ$

Example:

$A \rightarrow B$
$\Rightarrow AC \rightarrow BC$

---

3. Transitivity Rule

If:

X → Y
Y → Z

then:

X → Z

Example:

sid → dept
dept → hod
⇒ sid → hod

Derived Rules

- Union
- Decomposition
- Pseudotransitivity

📌 **Exam Note:**
Armstrong's axioms are **sound and complete**.

KEYS IN RELATIONAL DATABASES

Super Key

A set of attributes that can uniquely identify a tuple.

Example:

sid
sid, sname

Candidate Key

A **minimal superkey**.

Properties:

- Uniquely identifies tuples
- No proper subset is a key

Primary Key

A candidate key chosen by the designer.

Prime Attribute

An attribute that is part of **any candidate key**.

Non-Prime Attribute

An attribute not part of any candidate key.

WHY NORMALIZATION IS NEEDED

Problems in Un-Normalized Tables

- Data redundancy
- Update anomaly
- Insert anomaly
- Delete anomaly

Example of Anomalies

Employee(eid, ename, dept, dept_location)

If department location changes:

- Must update multiple rows → **update anomaly**

NORMAL FORMS

UN-NORMAL FORM (UNF)

Definition

A relation is in UNF if it contains:

- Repeating groups
- Multi-valued attributes

Example:

Student(sid, sname, courses)

FIRST NORMAL FORM (1NF)

Definition

A relation is in **1NF** if:

- All attributes contain **atomic values**
- No repeating groups

Example

❌ Not in 1NF:

Student(sid, sname, courses)
courses = {CSE101, CSE102}

☑ In 1NF:

Student(sid, sname, course)

SECOND NORMAL FORM (2NF)

Definition

A relation is in **2NF** if:

- It is in 1NF
- No **partial dependency** exists

What is Partial Dependency?

A non-prime attribute depends on **part of a composite key**.

Example

Enrollment(sid, cid, sname, cname)
Key: (sid, cid)

FDs:

sid → sname
cid → cname

Violation:

- sname depends on sid (partial key)
- cname depends on cid (partial key)

2NF Decomposition

Student(sid, sname)
Course(cid, cname)
Enrollment(sid, cid)

THIRD NORMAL FORM (3NF)

Definition

A relation is in **3NF** if:

- It is in 2NF
- No **transitive dependency** exists

What is Transitive Dependency?

If:

A → B
B → C

and B is **not a candidate key**, then C is transitively dependent on A.

Example

Employee(eid, ename, dept, dept_location)

FDs:

eid → dept
dept → dept_location

Transitive dependency:

eid → dept_location

3NF Decomposition

Employee(eid, ename, dept)
Department(dept, dept_location)

BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in **BCNF** if:

For every non-trivial FD **X** → **Y**, X must be a **superkey**.

Why BCNF is Stronger than 3NF

- Removes **all redundancy**
- Handles overlapping candidate keys better

Example (BCNF Violation)

R(A, B, C)
FDs:
A → B
C → B

Keys:

- AC is candidate key

Violation:

- C → B, but C is not a superkey

BCNF Decomposition

R1(C, B)
R2(A, C)

LOSSLESS AND DEPENDENCY PRESERVATION

Lossless Decomposition

A decomposition is **lossless** if joining decomposed relations gives the original relation **without losing data**.

Dependency Preservation

All functional dependencies can be enforced **without joining tables**.

📌 **Important Note**

- 3NF guarantees dependency preservation
- BCNF may lose dependency preservation

FINAL EXAM SUMMARY

- Functional dependency defines attribute relationships
- Attribute closure is key to finding candidate keys
- Normalization reduces redundancy and anomalies
- 1NF → atomic values
- 2NF → no partial dependency
- 3NF → no transitive dependency
- BCNF → determinant must be superkey
- BCNF is stronger but may sacrifice dependency preservation

Segment 6: Indexing and Hashing

WHY INDEXING IS NEEDED

Problem Without Index

When a database table is large, searching a record using a condition like:

SELECT * FROM Student WHERE sid = 105;

requires a **full table scan**, meaning:

- Every row is checked one by one
- Time complexity ≈ **O(n)**
- Very slow for large datasets

Solution: Indexing

An **index** is a data structure that allows the DBMS to **locate records quickly** without scanning the entire table.

📌 **Key idea:**
Index works like the **index of a book** → you don't read the whole book, you jump to the page.

WHAT IS AN INDEX?

Definition

An index is a **separate data structure** that stores:

- Search key value
- Pointer to the actual data record

General Structure

(search_key, pointer_to_record)

ADVANTAGES OF INDEXING

- Faster data retrieval
- Reduced disk I/O
- Efficient query execution
- Essential for large databases

DISADVANTAGES

- Extra storage required
- Slower INSERT, UPDATE, DELETE
- Index maintenance overhead

CLASSIFICATION OF INDEXES

Indexes are broadly classified into:

1. **Ordered Indexes**
2. **Hash Indexes**

ORDERED INDEXES

Definition

In ordered indexes, search keys are stored in **sorted order**.

PRIMARY INDEX

Definition

A **primary index** is created on the **primary key** of a table, and the data file itself is **sorted on that key**.

Characteristics

- One primary index per table
- Data is stored in sorted order
- Index entry per **block**, not per record

Example

Table sorted by sid:

Student(sid, sname, dept)

Index:

sid | block_pointer

📌 **Primary index is usually sparse.**

SECONDARY INDEX

Definition

A **secondary index** is created on a **non-primary attribute**.

Characteristics

- Data file is NOT sorted on the indexed attribute
- Multiple secondary indexes allowed
- Usually **dense**

Example

Index on dept:

dept | list_of_pointers

CLUSTERED INDEX

Definition

A clustered index is created on an attribute where **records with the same key value are stored together**.

Example

Employees grouped by department:

dept = CSE → stored together

📌 Only **one clustered index** per table.

DENSE INDEX

Definition

A dense index contains an **index entry for every search key value** (or every record).

Structure

key → record_pointer

Characteristics

- Faster search
- Larger index size
- Used when file is not ordered

Example

sid → pointer to record

📌 **Secondary indexes are dense by nature.**

SPARSE INDEX

Definition

A sparse index contains index entries for **only some search key values**, usually **one per data block**.

Characteristics

- Smaller size
- Requires ordered data file
- Slower than dense index

Example

first_sid_in_block → block_pointer

📌 **Primary indexes are usually sparse.**

DENSE vs SPARSE INDEX (EXAM FAVORITE)

Feature	Dense Index	Sparse Index
Entries	One per record	One per block
Size	Larger	Smaller
Search speed	Faster	Slightly slower
Data ordering	Not required	Required
Use case	Secondary index	Primary index

MULTILEVEL INDEXING

Problem with Single-Level Index

- Large index → many disk accesses

Solution

Create an **index on the index itself**

Definition

A **multilevel index** is an index where:

- First level indexes the data
- Higher levels index the lower index

Structure

Top-level index
↓
Lower-level index
↓
Data file

Benefit

- Reduces disk access from **O(n)** to **O(log n)**

📌 **B+ Tree is the most common multilevel index.**

B+ TREE INDEX FILES

WHAT IS A B+ TREE?

Definition

A **B+ tree** is a balanced multi-level index structure where:

- All actual data pointers are stored at **leaf nodes**
- Internal nodes contain only keys

Properties of B+ Tree

- Always balanced
- All leaf nodes at same level
- Leaf nodes are linked (good for range queries)

Order of B+ Tree

If order = n:

- Internal node can have **at most n children**
- At least [n/2] children

Why B+ Tree is Used

- Efficient search
- Efficient insertion and deletion
- Excellent for range queries

📌 **Exam Tip:**
B+ trees minimize disk I/O.

INSERTION IN B+ TREE (CONCEPT)

Steps:

1. Insert key in leaf node
2. If overflow occurs:
    - Split the node
    - Promote middle key
3. Repeat upward if needed

DELETION IN B+ TREE (CONCEPT)

Steps:

1. Delete key from leaf
2. If underflow occurs:
    - Borrow from sibling OR
    - Merge nodes
3. Adjust parent keys

HASH INDEXING

WHAT IS HASHING?

Definition

Hashing maps a search key to a **bucket** using a **hash function**.

h(key) → bucket_address

HASH FUNCTION

Definition

A hash function converts a search key into a bucket number.

Example

h(x) = x mod 5

If x = 12:

12 mod 5 = 2

📌 **Good hash function distributes keys uniformly.**

STATIC HASHING

Definition

In static hashing:

- Number of buckets is fixed
- Hash function does not change

Problem

- Bucket overflow
- Performance degrades over time

BUCKET OVERFLOW

Why Overflow Happens

- Too many keys map to same bucket

OVERFLOW HANDLING TECHNIQUES

---

1. Overflow Chaining

- Extra overflow bucket created
- Linked to original bucket

Bucket → Overflow → Overflow

📌 **Used in exams frequently**

2. Closed Hashing (Open Addressing)

Records are stored **inside the hash table itself**.

Linear Probing

h(k), h(k)+1, h(k)+2 ...

Problem:

- Primary clustering

Quadratic Probing

$h(k) + i^2$

Reduces clustering.

OPEN HASHING

Definition

Each bucket contains a **linked list** of records.

📌 Also called **separate chaining**.

STATIC vs DYNAMIC HASHING

Feature	Static Hashing	Dynamic Hashing
Buckets	Fixed	Can grow/shrink
Overflow	Frequent	Rare
Performance	Degrades	Stable
Example	Simple hashing	Extendible hashing

DYNAMIC HASHING (CONCEPT)

Goal

- Adjust bucket size dynamically
- Reduce overflow

Examples

- Extendible hashing
- Linear hashing

📌 Usually **theory-based exam questions**.

INDEXING vs HASHING (VERY IMPORTANT)

Feature	Indexing	Hashing
Best for	Range queries	Exact match
Order	Sorted	Unordered
Structure	Tree-based	Hash-based
Example	B+ Tree	Hash index

WHEN TO USE WHAT (EXAM ANSWER)

- Use **B+ tree indexing** for:
  - Range queries
  - Sorting
  - Inequality conditions
- Use **hashing** for:
  - Equality search (=)
  - Fast exact lookup

FINAL EXAM SUMMARY (SEGMENT 6)

- Index improves search speed
- Primary index → sparse
- Secondary index → dense
- Multilevel index reduces disk I/O
- B+ tree is most efficient index structure
- Hashing uses hash function

- Overflow handled by chaining or probing
- Static hashing has fixed buckets
- Dynamic hashing adapts to data growth

Segment 7: Transaction

INTRODUCTION TO TRANSACTIONS

What is a Transaction?

A **transaction** is a sequence of database operations that represents **one logical unit of work**.

These operations may include:

- Reading data
- Writing data
- Updating records
- Deleting records

Formal Definition

A transaction is a sequence of read and write operations that must be executed **completely or not at all**, in order to preserve database correctness.

Real-Life Example: Bank Transfer

Transfer 500 from Account A to Account B:

1. Read balance of A
2. A = A − 500
3. Write balance of A
4. Read balance of B
5. B = B + 500
6. Write balance of B

This entire sequence is **one transaction**.

If any step fails, the whole transaction must fail.

WHY TRANSACTIONS ARE NECESSARY

Without transactions:

- Partial updates can occur
- Database can become inconsistent
- Failures can corrupt data

Transactions ensure:

- Correctness
- Reliability
- Consistency
- Recovery from failures

ACID PROPERTIES

Every transaction must satisfy **ACID** properties.

---

1. ATOMICITY

Definition

Atomicity means that a transaction is **indivisible**:

Either **all operations execute**, or **none execute**.

There is **no partial execution**.

Example

If money is deducted from Account A but not added to Account B due to system failure, atomicity is violated.

How Atomicity is Implemented

- Undo logs
- Rollback mechanisms
- Shadow copy technique

📌 If a transaction fails → all changes are undone.

2. CONSISTENCY

Definition

Consistency ensures that a transaction takes the database from **one valid state to another valid state**.

It must satisfy:

- Integrity constraints
- Business rules
- Domain rules

Example

- Balance cannot become negative
- Foreign keys must refer to valid primary keys

📌 Consistency is the responsibility of **both DBMS and application logic**.

---

3. ISOLATION

Definition

Isolation ensures that **concurrent transactions do not interfere** with each other.

Each transaction behaves as if it is executing **alone**.

Problem Without Isolation

Concurrent execution may cause:

- Dirty reads
- Lost updates
- Inconsistent data

---

4. DURABILITY

Definition

Durability guarantees that once a transaction **commits**, its effects are **permanent**, even if:

- System crashes
- Power fails

How Durability is Implemented

- Write-ahead logging
- Stable storage
- Checkpoints

📌 Committed data must survive failures.

TRANSACTION STATES

A transaction moves through different **states** during execution.

TRANSACTION STATE DIAGRAM (CONCEPTUAL)

States of a Transaction

1. **Active**
2. **Partially Committed**
3. **Committed**
4. **Failed**
5. **Aborted**
6. **Terminated**

---

1. Active State

- Transaction has started
- Executing read/write operations

2. Partially Committed

- Last statement executed
- Waiting for final commit

---

3. Committed

- Transaction completed successfully
- Changes permanently stored

---

4. Failed

- Error occurs (system crash, constraint violation)

---

5. Aborted

- All changes rolled back
- Database restored to previous state

---

6. Terminated

- Transaction leaves system

📌 **Exam Tip:**
Always draw or describe the state diagram in order.

IMPLEMENTATION OF ATOMICITY AND DURABILITY

LOG-BASED RECOVERY

Basic Idea

All changes made by transactions are **recorded in a log file** before being applied to the database.

Log Entry Format

<Ti, X, old_value, new_value>

Why Logging is Important

- Undo incomplete transactions
- Redo committed transactions
- Recover from system crashes

UNDO and REDO

- **UNDO** → rollback failed transactions
- **REDO** → reapply committed transactions

SHADOW COPY TECHNIQUE

Concept

- Keep a shadow copy of the database
- Apply changes to a temporary copy
- Replace original only after commit

Advantages

- Simple concept
- Guarantees atomicity

Disadvantages

- High storage cost
- Not suitable for large databases

📌 Mostly **theoretical**, but exam-important.

CONCURRENT EXECUTION OF TRANSACTIONS

Why Concurrency is Needed

- Better CPU utilization
- Higher throughput
- Faster response time

But concurrency introduces **problems**.

PROBLEMS CAUSED BY CONCURRENCY

---

1. Lost Update Problem

Two transactions update the same data; one update is lost.

---

2. Dirty Read Problem

A transaction reads data written by another transaction that has **not committed yet**.

---

3. Unrepeatable Read

Same read gives different values due to concurrent update.

---

4. Inconsistent Analysis

Aggregate values computed on partially updated data.

SCHEDULES

What is a Schedule?

A **schedule** is the **order in which operations from multiple transactions are interleaved**.

Serial Schedule

- Transactions execute one after another
- No interleaving

Non-Serial Schedule

- Operations are interleaved
- Faster, but risky

SERIALIZABILITY

Definition

A schedule is **serializable** if its effect is equivalent to some **serial schedule**.

Why Serializability Matters

- Ensures correctness
- Allows concurrency without inconsistency

TYPES OF SERIALIZABILITY

---

1. CONFLICT SERIALIZABILITY

Definition

A schedule is conflict-serializable if it can be transformed into a serial schedule by **swapping non-conflicting operations**.

Conflicting Operations

Two operations conflict if:

- They belong to different transactions
- They operate on the same data item
- At least one is a write

Testing Conflict Serializability

Use a **precedence graph**.

Precedence Graph Rules

- Node → Transaction
- Edge Ti → Tj if:
    - Ti's operation conflicts and occurs before Tj's

📌 If graph has **no cycle**, schedule is conflict-serializable.

---

2. VIEW SERIALIZABILITY

Definition

A schedule is view-serializable if:

- Reads initial values correctly
- Reads final values correctly
- Writes final values correctly

Comparison

Feature	Conflict	View
Easy to test	Yes	No
Covers more schedules	No	Yes
Practical	Yes	Rare

RECOVERABILITY

RECOVERABLE SCHEDULE

Definition

A schedule is recoverable if:

A transaction commits **only after** all transactions whose data it read have committed.

Why Important

Prevents committing data based on uncommitted values.

CASCADING ROLLBACK

Occurs when:

- One rollback forces many other rollbacks

CASCADLESS SCHEDULE

Definition

A schedule is cascadeless if:

A transaction reads data **only after the writing transaction commits**.

Advantage

- No cascading rollback
- Safer execution

IMPLEMENTATION OF ISOLATION

Isolation is implemented using:

- Locks
- Timestamps
- Protocols (covered in Segment 8)

TESTING SERIALIZABILITY

Steps

1. Identify conflicting operations
2. Build precedence graph
3. Check for cycles
4. Determine serial order

📌 This is a **very common exam question**.

FINAL EXAM SUMMARY (SEGMENT 7)

- Transaction = logical unit of work
- ACID ensures correctness
- Atomicity → all or nothing

- Consistency → valid states
- Isolation → no interference
- Durability → permanence
- Logs and shadow copy ensure recovery
- Concurrency improves performance but causes problems
- Serializability guarantees correctness
- Precedence graph tests conflict serializability
- Cascadeless schedules prevent cascading rollback

Segment 8: Concurrency Control, Recovery System and Distributed Databases

---

PART A: CONCURRENCY CONTROL

WHY CONCURRENCY CONTROL IS NEEDED

When multiple transactions execute **simultaneously**, concurrency improves:

- System throughput
- CPU utilization
- Response time

However, without control, concurrency can cause:

- Inconsistent database states
- Lost updates
- Dirty reads
- Cascading rollbacks

👉 **Concurrency control ensures correctness while allowing parallel execution.**

LOCK-BASED CONCURRENCY CONTROL

LOCKS

What is a Lock?

A **lock** is a variable associated with a data item that controls **which transaction can access the data**.

Types of Locks

1. Shared Lock (S-lock)

- Used for **read** operations
- Multiple transactions can hold S-lock simultaneously

2. Exclusive Lock (X-lock)

- Used for **write** operations
- Only one transaction can hold X-lock

Lock Compatibility Table

Lock Held	S-lock Request	X-lock Request
S-lock	✔ Allowed	✘ Not allowed
X-lock	✘ Not allowed	✘ Not allowed

LOCKING PROTOCOL

A **locking protocol** defines **rules** that transactions must follow when requesting and releasing locks.

TWO-PHASE LOCKING (2PL) PROTOCOL

Definition

The **Two-Phase Locking (2PL)** protocol ensures **conflict serializability** by dividing transaction execution into two phases.

Two Phases

1. Growing Phase

- Transaction can **acquire locks**
- Cannot release any lock

2. Shrinking Phase

- Transaction can **release locks**

- Cannot acquire any new lock

Key Rule

Once a transaction releases a lock, it cannot request any new locks.

Why 2PL Works

- Prevents cyclic dependencies
- Guarantees conflict-serializable schedules

📌 **Important Exam Point:**
2PL ensures **serializability**, but **not freedom from deadlock**.

VARIANTS OF 2PL

Strict 2PL

- X-locks are released only after commit or abort
- Prevents dirty reads
- Produces cascadeless schedules

Rigorous 2PL

- Both S-lock and X-lock are released after commit
- Strongest form of locking

DEADLOCK

WHAT IS DEADLOCK?

A **deadlock** occurs when:

Two or more transactions wait indefinitely for each other to release locks.

Deadlock Example

- T1 holds lock on A, waits for B
- T2 holds lock on B, waits for A

Neither can proceed.

DEADLOCK HANDLING METHODS

---

1. DEADLOCK PREVENTION

Prevention avoids deadlocks by **restricting lock requests**.

WAIT–DIE SCHEME

- Older transaction waits
- Younger transaction aborts

Rule:

- If Ti requests a lock held by Tj:
  - If Ti is older → wait
  - If Ti is younger → die (abort)

WOUND–WAIT SCHEME

- Older transaction wounds (forces rollback of) younger
- Younger waits if older requests lock

---

2. DEADLOCK DETECTION

Wait-For Graph

- Nodes = transactions
- Edge Ti → Tj means Ti is waiting for Tj

📌 **Cycle in graph = deadlock**

---

3. DEADLOCK RECOVERY

- Abort one or more transactions
- Release locks
- Restart transaction

GRAPH-BASED PROTOCOL

Definition

Graph-based protocol uses a **directed acyclic graph (DAG)** to determine lock acquisition order.

Rule

- A transaction can lock X only if it has locked all predecessor items of X.

📌 Prevents deadlock by design.

TREE PROTOCOL

Definition

Data items are organized as a **tree**.

Rules

1. First lock can be on any node
2. Subsequently, transaction must lock parent before child
3. Locks can be released anytime
4. Once released, cannot be relocked

📌 Ensures serializability and deadlock freedom.

TIMESTAMP-BASED PROTOCOLS

TIMESTAMP

Definition

A **timestamp** is a unique number assigned to a transaction when it starts.

Used to determine **transaction order**.

Timestamp Ordering Protocol

Rules:

- If a transaction violates timestamp order → it is rolled back

Ensures:

- Serializability
- No deadlock

📌 Trade-off: more rollbacks possible.

PART B: RECOVERY SYSTEM

FAILURE CLASSIFICATION

Types of Failures

1. **Transaction Failure**
    - Logical error
    - Constraint violation
2. **System Failure**
    - Power failure
    - OS crash
3. **Media Failure**
    - Disk crash
    - Data loss

STORAGE TYPES

Volatile Storage

- Main memory (RAM)
- Data lost on crash

Non-Volatile Storage

- Disk storage
- Survives system crash

Stable Storage

- Extremely reliable
- Used for logs

CHECKPOINTS

What is a Checkpoint?

A **checkpoint** is a mechanism where:

- All modified data is written to disk
- Log records are synchronized

Why Checkpoints Are Needed

- Reduce recovery time
- Limit amount of log scanning

Checkpoint Procedure

1. Suspend transactions temporarily
2. Write dirty buffers to disk
3. Write checkpoint record to log
4. Resume transactions

RECOVERY USING LOGS

- UNDO for uncommitted transactions
- REDO for committed transactions

📌 Log-based recovery + checkpoints = fast recovery.

PART C: DISTRIBUTED DATABASES

WHAT IS A DISTRIBUTED DATABASE?

A **distributed database** is a collection of logically related data stored across **multiple physical locations**, connected by a network.

Why Distributed Databases?

- Improved reliability
- Better performance
- Scalability
- Local autonomy

DISTRIBUTED DATA CONCEPTS

FRAGMENTATION

Fragmentation divides data into pieces.

Types of Fragmentation

1. Horizontal Fragmentation

- Divide rows

Student_CSE
Student_EEE

---

2. Vertical Fragmentation

- Divide columns

Student(sid, sname)
Student(sid, dept)

---

3. Hybrid Fragmentation

- Combination of horizontal and vertical

REPLICATION

Definition

Replication means **storing copies of data at multiple sites**.

Advantages

- Faster access
- Fault tolerance
- High availability

Disadvantages

- Update complexity
- Consistency issues

DISTRIBUTED TRANSACTIONS (OVERVIEW)

- Require coordination across sites
- Two-phase commit protocol used (conceptual)
- Ensures atomicity across locations

FINAL EXAM SUMMARY (SEGMENT 8)

- Concurrency control ensures correctness
- Locks manage shared data access
- 2PL ensures serializability
- Deadlocks must be handled
- Timestamp protocols avoid deadlock
- Recovery uses logs and checkpoints
- Distributed DB improves scalability
- Fragmentation improves performance
- Replication improves availability